

2) Solution Overview

Last updated by | Mennell, Jacob | 21 May 2026 at 10:44 BST

Solution Overview

Purpose

This page summarises the main parts of the Claims Augmented Triage solution and how they work together to support ingestion, retrieval, AI-assisted processing, and user-facing delivery.

Solution Summary

Claims Augmented Triage is a multi-service platform that brings together document ingestion, retrieval-ready storage, AI workflow orchestration, and browser-based access in a single claims solution. The solution is structured as a small set of focused services:

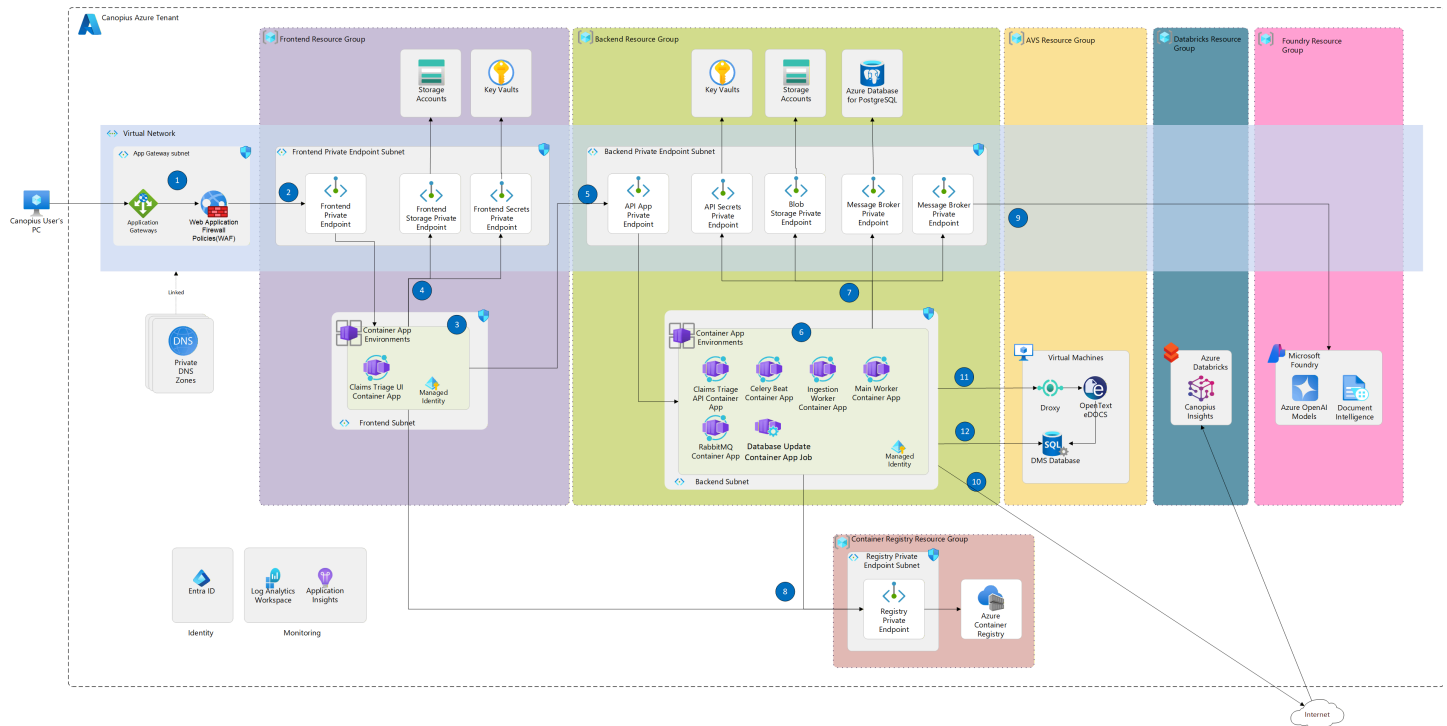
- an ingestion worker that transforms documents into structured chunks and embeddings
- a main worker that runs extraction and checking workflows over claim evidence
- an API service that exposes operational and user-facing endpoints
- a frontend application that presents claim-centric views and task outputs
- shared data and messaging infrastructure that supports asynchronous execution and retrieval

High-Level Platform Architecture

The following infrastructure diagram shows how the solution is deployed within Azure, including network segmentation, private endpoints, container workloads, messaging components, AI integrations, and supporting platform services.

It illustrates:

- separation between frontend, backend, registry, and analytics resource groups
- use of private networking and private endpoints for secure internal communication
- container-based execution using Azure Container Apps
- integration with PostgreSQL, RabbitMQ, OpenText DMS, Azure OpenAI / Foundry, and Databricks
- secure ingress through Application Gateway and WAF
- managed identity usage for service-to-service authentication



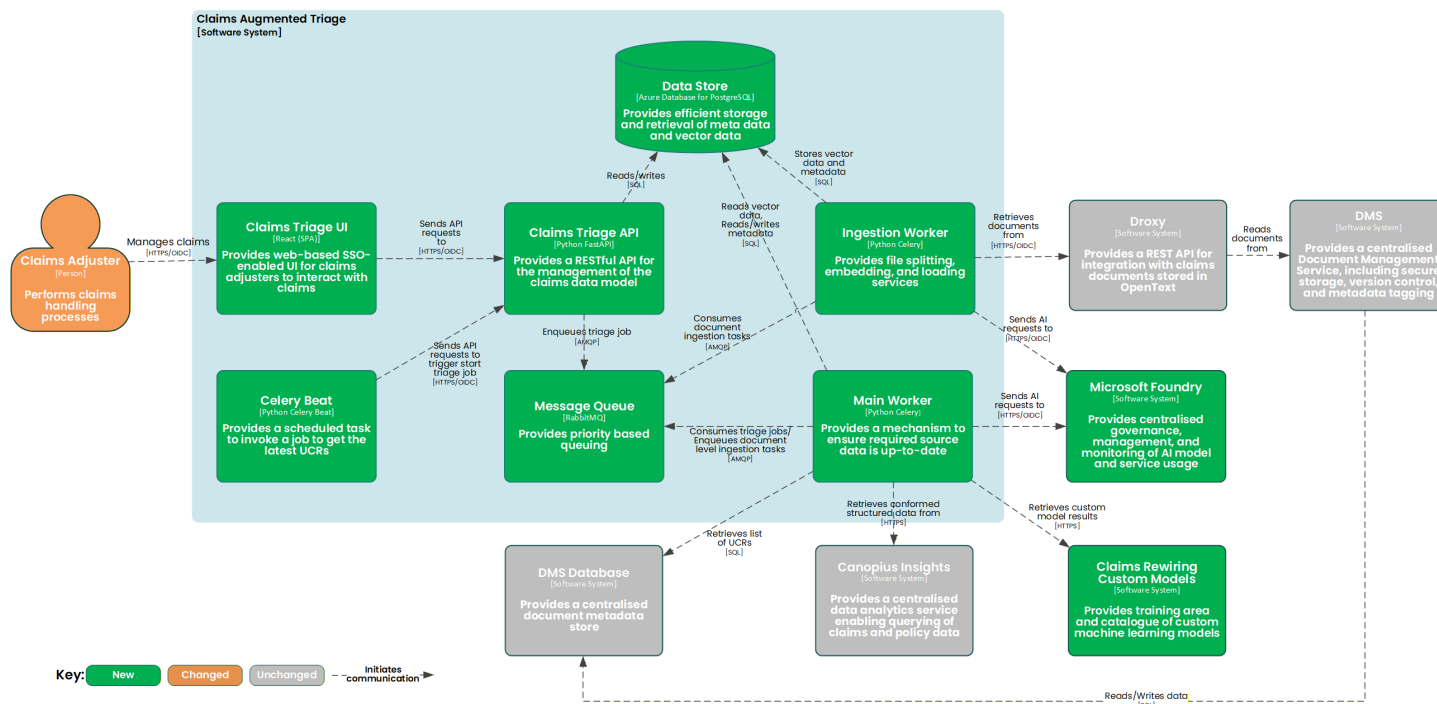
The architecture is designed to isolate responsibilities across network and service boundaries while allowing the ingestion, orchestration, retrieval, and presentation layers to communicate through secured internal channels.

Logical Solution Architecture

The following logical architecture diagram focuses on the runtime interactions between the major software components and external systems involved in claims processing workflows.

It shows:

- how claims adjusters interact with the platform through the Claims Triage UI
- how the API coordinates orchestration requests
- how asynchronous workloads are routed through RabbitMQ
- the relationship between ingestion, retrieval, orchestration, and AI execution
- integration points with DMS, Insights, and custom AI models
- where AI-assisted retrieval and reasoning occur within the workflow



This view complements the infrastructure diagram by focusing on application responsibilities and runtime data flow rather than deployment topology.

Where AI Is Used

AI is used in three main areas:

1. Document ingestion

DI-based parsing is used to reconstruct text, detect structure, preserve header hierarchy, and enrich chunks with layout-related metadata.

The ingestion pipeline converts raw claim documents into retrieval-ready representations by:

- extracting structured text and tables
- preserving document hierarchy and layout semantics
- generating embeddings for semantic retrieval
- storing enriched chunk metadata for downstream orchestration

2. Retrieval-backed reasoning

Retrieved claim evidence is assembled from stored chunks using lexical, semantic, and hybrid retrieval patterns, with optional reranking layers.

The retrieval layer supports:

- semantic similarity search using vector embeddings
- keyword and metadata-based retrieval
- hybrid retrieval strategies
- contextual assembly of evidence before LLM execution

- reranking for higher precision task execution

3. Orchestrated task execution

The main worker runs extraction and check flows with explicit routing, QA stages, retries, and specialised execution modes such as progressive-stop and progressive-fanin.

This orchestration layer is responsible for:

- coordinating AI execution flows
 - applying retry and validation policies
 - handling task fan-out and aggregation
 - managing execution state and workflow progression
 - shaping final outputs returned to users and downstream systems
-

Core Solution Building Blocks

Frontend application

The frontend is a React application using route-based navigation for the home page, claim pages, extracted information views, generic checks, model-specific pages, and administration.

The UI layer provides:

- SSO-enabled access for claims users
- claim-centric workflow navigation
- review and validation interfaces
- operational visibility into extraction and checking results
- administration and configuration tooling

API service

The API is a FastAPI application mounted under `/api/` and organised into route groups for claims, orchestration, models, documents, feedback, and computation. Authentication and role checks are enforced through JWT-based request handling.

The API acts as the central coordination layer between:

- frontend experiences
- orchestration workflows
- retrieval services
- persistence components
- background worker execution

Ingestion worker

The ingestion worker transforms raw documents into retrieval-ready assets. It performs document parsing, markdown reconstruction, header-aware chunking, table-aware chunk handling, embedding generation, and persistence into the chunk store.

The ingestion process is intentionally isolated from runtime orchestration so that:

- ingestion workloads can scale independently
- retrieval quality can evolve separately from orchestration logic
- document processing pipelines can be updated without affecting runtime APIs

Main worker

The main worker contains the runtime logic for AI-assisted extraction and checking. It coordinates retrieval, execution routing, QA, retries, and output shaping.

The worker layer supports:

- asynchronous execution
- long-running AI workflows
- queue-driven processing
- task prioritisation
- distributed execution patterns

Shared libraries

The shared libraries provide common database models, schemas, and retrieval functionality. In particular, the retrieval library contains the core lexical, semantic, hybrid, and reranking logic used by the orchestration layer.

This shared-library approach helps:

- reduce duplication across services
- standardise retrieval behaviour
- simplify schema evolution
- maintain consistent orchestration contracts

Data and messaging layer

PostgreSQL with pgvector is used as both the operational persistence layer and the vector-backed retrieval store. RabbitMQ provides the asynchronous messaging backbone for worker-based execution.

Together, these components provide:

- operational persistence
- vector similarity retrieval
- durable asynchronous execution
- task prioritisation and routing
- decoupled worker coordination

High-Level Runtime Model

At a high level, the solution behaves as follows:

1. documents and claim-related inputs enter the platform
2. ingestion transforms raw content into structured chunks plus embeddings
3. chunks and metadata are persisted in the database
4. AI workflows retrieve evidence from the stored chunk set
5. extraction or check logic executes with QA and bounded retries
6. results are exposed through the API and consumed by the frontend

Operationally, this enables:

- asynchronous document processing
- scalable retrieval-backed AI execution
- separation between ingestion and runtime workloads
- controlled orchestration and validation of AI outputs
- secure enterprise integration with upstream and downstream systems

Why The Architecture Is Split This Way

The solution is structured with a deliberate separation of concerns:

- ingestion concerns are isolated from orchestration concerns
- retrieval logic is centralised in a reusable library rather than duplicated across services
- API delivery is separated from background workflow execution
- frontend concerns are decoupled from backend implementation details
- infrastructure boundaries align with operational and security responsibilities

This architecture enables:

- independent scaling of ingestion and orchestration workloads
- clearer operational ownership
- stronger security isolation
- easier testing and deployment
- reduced coupling between runtime components
- future extensibility for additional AI workflows and retrieval strategies

The combination of infrastructure isolation, asynchronous execution, retrieval-backed orchestration, and modular service boundaries provides a scalable foundation for AI-assisted claims processing workflows.